Experiment No. 4

Aim

Implement Preemptive Shortest Remaining Time First (SRTF) CPU Scheduling Algorithm to evaluate turnaround time and waiting time for a given problem.

Theory

Shortest Remaining Time First (SRTF) is the preemptive version of the Shortest Job First scheduling algorithm. At every point in time, the process with the smallest remaining burst time (the CPU time still required to complete it) is given the CPU. Unlike non-preemptive SJF, a currently running process can be interrupted (preempted) if a new process arrives whose burst time is shorter than the current process's remaining execution time - the newly arrived process then takes over the CPU immediately.

SRTF is typically simulated by processing the schedule in discrete time units (unit-time simulation): at each time step, the scheduler checks all processes that have arrived so far and are not yet completed, selects the one with the smallest remaining burst time, executes it for one unit of time, and decrements its remaining time. This is repeated until all processes have completed, with the scheduler re-evaluating its choice at every time unit, allowing it to switch to a newly arrived shorter process mid-execution.

Because it always favours the job closest to completion, SRTF achieves an even lower average waiting time than non-preemptive SJF for the same set of processes. However, this comes at the cost of frequent context switching, which introduces real-world overhead (not captured in simple simulations), and it can cause starvation of longer processes if a steady stream of short processes continues to arrive, similar to non-preemptive SJF but more severe due to preemption.

For each process, the key performance metrics are computed as:

- Completion Time (CT): the time at which a process finally finishes execution (after possibly being preempted and resumed multiple times).
- Turnaround Time (TAT): TAT = Completion Time − Arrival Time, the total time from arrival to completion.
- Waiting Time (WT): WT = Turnaround Time − Burst Time, the total time spent waiting in the ready queue across all the periods the process was not running.

The average turnaround time and average waiting time are computed by summing these values across all processes and dividing by the total number of processes, exactly as in non-preemptive scheduling, but the values themselves reflect the effect of preemption on each process's execution.

Code

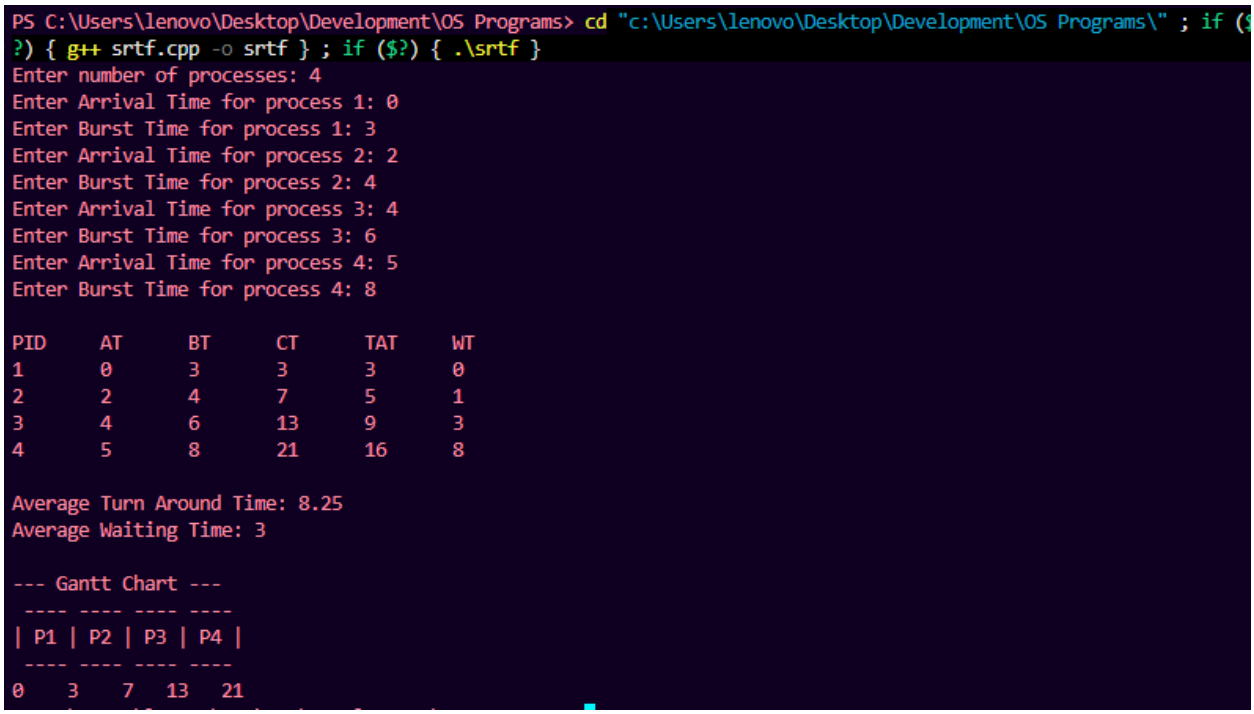
```
#include <iostream>
#include <vector>
#include <climits>
#include <string>
```

```
using namespace std;

struct Process {
    int id;         // Process ID
    int at;         // Arrival Time
    int bt;         // Burst Time
    int rt;         // Remaining Time
    int ct;         // Completion Time
    int tat;        // Turn Around Time
    int wt;         // Waiting Time
    bool started;   // Has the process started execution?
};

struct GanttRecord {
    string name;
    int start_time;
    int end_time;
};

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    vector<Process> p(n);
    for (int i = 0; i < n; ++i) {
        p[i].id = i + 1;
        cout << "Enter Arrival Time for process " << p[i].id << ": ";
        cin >> p[i].at;
        cout << "Enter Burst Time for process " << p[i].id << ": ";
        cin >> p[i].bt;
        p[i].rt = p[i].bt;
        p[i].ct = p[i].tat = p[i].wt = 0;
        p[i].started = false;
    }

    int currentTime = 0;
    int completed = 0;
    vector<bool> done(n, false);

    vector<GanttRecord> gantt;
    while (completed < n) {
        int minRt = INT_MAX;
        int select = -1;

        for (int i = 0; i < n; ++i) {
            if (!done[i] && p[i].at <= currentTime && p[i].rt > 0) {
                if (p[i].rt < minRt) {
                    minRt = p[i].rt;
                    select = i;
```

```
            }
        }
    }

    string currentName = (select == -1) ? "IDLE" : "P" + to_string(p[select].id);

    if (gantt.empty() || gantt.back().name != currentName) {
        gantt.push_back({currentName, currentTime, currentTime + 1});
    } else {
        gantt.back().end_time = currentTime + 1;
    }

    if (select == -1) {
        currentTime++;
        continue;
    }
    if (!p[select].started) {
        p[select].started = true;
    }

    p[select].rt--;
    currentTime++;
    if (p[select].rt == 0) {
        p[select].ct = currentTime;
        p[select].tat = p[select].ct - p[select].at;
        p[select].wt = p[select].tat - p[select].bt;
        done[select] = true;
        completed++;
    }
}

// Output table
cout << "\nPID\tAT\tBT\tCT\tTAT\tWT\n";
long long totalTAT = 0, totalWT = 0;
for (int i = 0; i < n; ++i) {
    cout << p[i].id << "\t"
         << p[i].at << "\t"
         << p[i].bt << "\t"
         << p[i].ct << "\t"
         << p[i].tat << "\t"
         << p[i].wt << "\n";
    totalTAT += p[i].tat;
    totalWT += p[i].wt;
}

cout << "\nAverage Turn Around Time: " << (double)totalTAT / n << "\n";
cout << "Average Waiting Time: " << (double)totalWT / n << "\n";

// Output Gantt Chart
cout << "\n--- Gantt Chart ---\n";
```

```
    // Print top bar
    cout << " ";
    for (const auto& g : gantt) {
        for (int i = 0; i < g.name.length() + 2; ++i) cout << "-";
        cout << " ";
    }
    cout << "\n|";

    // Print processes / idle states
    for (const auto& g : gantt) {
        cout << " " << g.name << " |";
    }
    cout << "\n ";

    // Print bottom bar
    for (const auto& g : gantt) {
        for (int i = 0; i < g.name.length() + 2; ++i) cout << "-";
        cout << " ";
    }
    cout << "\n";

    // Print timelines
    cout << gantt[0].start_time;
    for (const auto& g : gantt) {
        // Calculate appropriate spacing based on string lengths
        int spaces = g.name.length() + 3 - to_string(g.end_time).length();
        if (spaces < 1) spaces = 1;
        cout << string(spaces, ' ') << g.end_time;
    }
    cout << "\n";

    return 0;
}
```

Output



Viva Questions

Q1. How does SRTF differ from non-preemptive SJF?

Ans. In non-preemptive SJF, once a process starts running, it continues until it completes, regardless of what arrives afterward. In SRTF, the CPU can be taken away from a currently running process if a new process arrives with a shorter remaining burst time, so the scheduler continuously re-evaluates which process has the least remaining work at every point in time.

Q2. Why does SRTF generally give a lower average waiting time than non-preemptive SJF?

Ans. SRTF always ensures the process closest to completion (i.e., with the smallest remaining burst time) is running at any given moment, immediately switching to a newly arrived shorter process instead of waiting for the current one to finish. This tighter, continuous optimization reduces the cumulative waiting experienced by processes compared to non-preemptive SJF, which commits to a process once it starts.

Q3. What is the main real-world overhead introduced by SRTF that simple simulations do not capture?

Ans. SRTF requires frequent context switching, since the CPU can be preempted every time a shorter process arrives. Each context switch has a real cost — saving and restoring process state/registers - which is not accounted for in simplified turnaround/waiting time calculations but adds actual overhead in a real operating system.

Q4. How is a process's remaining burst time tracked during SRTF scheduling?

Ans. Each process maintains a remaining time value, initialized to its original burst time. As the process executes (in the unit-time or event-driven simulation), its remaining time is decremented accordingly. The scheduler compares these remaining time values across all arrived, incomplete processes at every scheduling decision to choose which one runs next.

Q5. Can starvation occur in SRTF, and how does it compare to starvation in non-preemptive SJF?

Ans. Yes, starvation can occur in SRTF if a continuous stream of processes with very short burst times keeps arriving, causing a process with a large burst time to be repeatedly preempted and never allowed to finish. This effect is generally more pronounced in SRTF than in non-preemptive SJF, since SRTF can interrupt a long process mid-execution, whereas non-preemptive SJF only delays a long process from starting in the first place.